

RideSmart: Modelagem e Análise de Rotas Urbanas com Trade-off entre Caminhada e Carro

Matheus Fernandes

Departamento de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil

Thales Varela

Departamento de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil

Resumo—Este trabalho apresenta o RideSmart, uma modelagem em grafos do problema de escolher o melhor ponto de embarque P em aplicações de mobilidade urbana. Dados origem A , destino B e raio máximo de caminhada X , comparamos cinco cenários de decisão e quatro algoritmos clássicos de caminho mínimo — Dijkstra simples, Dijkstra com Min-Heap, A* com heurística great-circle e Dijkstra Bidirecional — aplicados à malha viária real de Coophab e Cajupiranga (Parnamirim/RN), extraída do OpenStreetMap. Introduzimos um modelo de trânsito sintético calibrado por tipo de via, um modelo monetário baseado em tarifas reais de *ride-hailing*, e a noção de paralelismo entre caminhada e tempo de busca do motorista. Os resultados quantificam o *trade-off* entre caminhar e chegar mais rápido, mostrando reduções de até 13,6% no tempo total e R\$ 1,19 de economia no preço da corrida. A análise revela ainda que o A* expande até 70% menos nós que o Dijkstra simples, que o Dijkstra Bidirecional supera o A* em pares longos, e que a saturação topológica limita o ganho da caminhada em certas configurações.

Index Terms—caminho mínimo, OSMnx, Dijkstra, A*, mobilidade urbana, otimização, grafos

I. INTRODUÇÃO

Aplicativos de mobilidade urbana como Uber e 99 enfrentam diariamente o problema de roteamento ótimo em malhas viárias densas e dinâmicas. Decisões sobre qual rua tomar, considerando trânsito e custo, dependem de algoritmos eficientes de caminho mínimo. Um aspecto pouco explorado é a **escolha do ponto de embarque**: em vez de chamar o motorista para o local exato do usuário (origem A), pode ser vantajoso caminhar até um ponto vizinho P de onde a rota até o destino B é mais rápida ou mais barata.

Este trabalho formaliza o problema **RideSmart**: dados A , B e uma distância máxima de caminhada X , encontrar P tal que a rota composta

$$A \xrightarrow{\text{caminhada} \leq X} P \xrightarrow{\text{carro}} B$$

minimize o tempo total ou o custo monetário da viagem.

Contribuições deste trabalho:

- 1) Modelagem em três pesos (`length`, `travel_time`, `travel_time_traffic`) sobre malha viária real (OSMnx);
- 2) Modelo de trânsito sintético em camadas, calibrado por tipo de via;
- 3) **Modelo de tempo de busca do motorista**, que captura o paralelismo “caminhar enquanto o Uber chega”;

- 4) Implementação *do zero* de quatro algoritmos clássicos de caminho mínimo, com análise comparativa de eficiência;
- 5) Quantificação do ganho — em segundos e em reais — gerado pela decisão ótima de P .

A Seção II descreve a modelagem do problema. A Seção III apresenta os quatro algoritmos. A Seção IV detalha os experimentos. A Seção V apresenta os resultados. A Seção VI discute criticamente os achados. A Seção VII conclui.

II. MODELAGEM DO PROBLEMA

A. Construção do Grafo

Utilizamos a biblioteca OSMnx para extrair a malha viária da região centrada em $(-5,909^\circ, -35,206^\circ)$ (próximo ao Parque das Árvores, Parnamirim/RN), em raio de 3 km com `network_type="drive"`. O grafo direcionado resultante contém 1342 nós e 3229 arestas. Convertemos para grafo simples não-direcionado G , preservando a aresta de menor comprimento entre cada par de nós, resultando em 1342 nós e 1936 arestas em um único componente conectado.

B. Pesos das Arestas

Cada aresta carrega três pesos:

- `length` (m): comprimento real da rua;
- `travel_time` (s): tempo em condições livres, calculado pelo OSMnx via velocidades padrão por tipo de via;
- `travel_time_traffic` (s): tempo com trânsito sintético (descrito a seguir).

C. Modelo de Trânsito Sintético

Para simular hora de pico, aplicamos:

$$\text{travel_time_traffic} = \text{travel_time} \times \mu_{\text{tipo}} \times \varepsilon \quad (1)$$

onde μ_{tipo} penaliza vias arteriais ($\mu = 4,0$ para *primary*, $\mu = 2,8$ para *secondary*, $\mu = 1,3$ para *residential*) e $\varepsilon \sim \mathcal{U}(0,9; 2,2)$ com *seed* fixa para reprodutibilidade. O fator médio resultante é $2,62\times$ com cauda chegando a $9\times$ em vias arteriais sob ruído máximo (Fig. 1). Esse modelo reflete o comportamento esperado: avenidas como a Av. Olavo Montenegro — divisor norte-sul da região — ficam significativamente mais lentas que ruas residenciais paralelas.

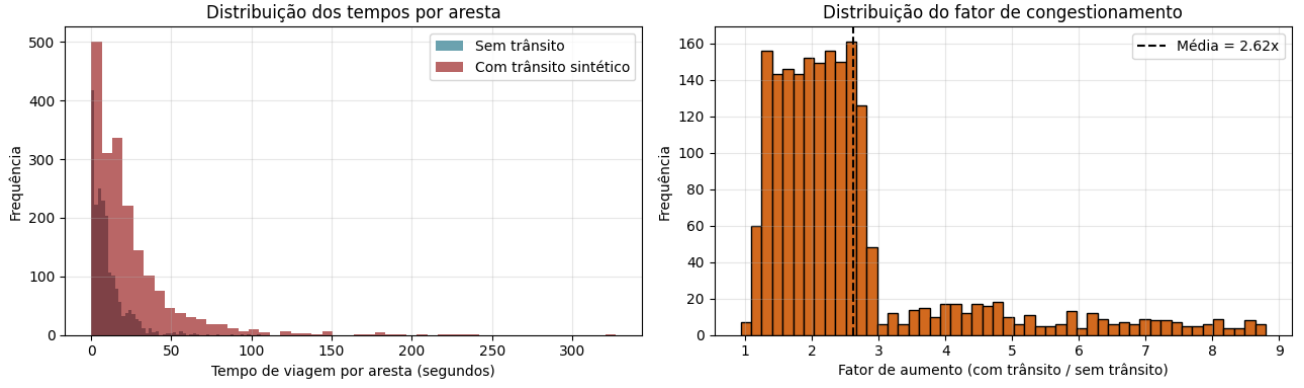


Figura 1. Distribuição dos tempos de viagem por aresta antes e depois do trânsito sintético (esquerda); distribuição do fator de aumento por aresta (direita). Média de $2,62\times$ com cauda em vias arteriais.

D. Modelo Monetário

Inspirado em tarifas reais de aplicativos de *ride-hailing* no Brasil:

$$C(d) = \left(R\$ 2,50 + \frac{d}{1000} \cdot R\$ 2,00 \right) \cdot s \quad (2)$$

onde d é a distância em metros dirigida pelo carro e $s \in \{1,0; 1,5\}$ é o multiplicador de *surge*, aplicado em cenários com trânsito. Cada metro caminhado é um metro a menos dirigido pelo carro, economizando R\$ 0,002/m (R\$ 0,003/m com *surge*).

E. Tempo de Busca do Motorista (Insight Central)

Em aplicações reais, o motorista demora um tempo τ_{uber} para chegar ao ponto P . Enquanto isso, o usuário pode caminhar em paralelo. Modelamos esse paralelismo como:

$$t_{\text{efetivo}}(\text{walk}) = \max(0, t_{\text{walk}} - \tau_{\text{uber}}) \quad (3)$$

Adotamos $\tau_{\text{uber}} = 300$ s (5 minutos, típico em regiões periféricas). Caminhar até ~ 417 m é, portanto, **efetivamente gratuito**, pois o usuário estaria aguardando o motorista de qualquer forma. Esse modelo é a chave para que a otimização produza soluções não-triviais ($P \neq A$).

F. Definição Formal

Sejam $W(u, v)$ o peso considerado, $d_W(u, v)$ o caminho mínimo segundo W , e $N(A, X) = \{v \in V : d_{\text{length}}(A, v) \leq X\}$ a vizinhança caminhável de A . O problema RideSmart busca:

$$P^* = \arg \min_{P \in N(A, X)} [\max(0, t_{\text{walk}}(A, P) - \tau_{\text{uber}}) + t_{\text{carro}}(P, B)] \quad (4)$$

Comparamos cinco cenários:

- 1) $W = \text{length}$ — menor distância total;
- 2) $W = \text{travel_time}$ — mais rápido sem trânsito;
- 3) $W = \text{travel_time_traffic}$ — mais rápido com trânsito (cenário principal);
- 4) $X = 0$ — baseline sem caminhada ($P = A$);
- 5) Diferença (4) — (3) — ganho efetivo de caminhar.

III. ALGORITMOS IMPLEMENTADOS

A. Min-Heap como Fila de Prioridade

Implementamos uma classe `MinHeap` como array dinâmico, com operações `push`, `pop` e `peek` seguindo a convenção pai = $(i-1)/2$, filhos = $2i+1$ e $2i+2$. As operações `push` e `pop` executam em $O(\log n)$ via *sift-up* e *sift-down*.

B. Dijkstra Simples — $O(V^2 + E)$

Versão didática que busca linearmente o vértice não-visitado de menor distância em cada iteração. Sem estrutura de prioridade. Útil como *baseline* para medir o impacto da Min-Heap.

C. Dijkstra com Min-Heap — $O((V + E) \log V)$

Substitui a busca linear pela operação `pop` da Min-Heap. Reduz o custo total de $O(V^2)$ para $O((V + E) \log V)$, dramaticamente mais rápido em grafos grandes.

D. A^* — Busca Guiada por Heurística

Combina o custo real $g(n)$ com uma estimativa heurística $h(n)$ via $f(n) = g(n) + h(n)$. Empregamos a distância *great-circle* entre os pontos n e *target*:

$$h(n) = 2R \cdot \arcsin \left(\sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_n \cos \varphi_t \sin^2 \frac{\Delta\lambda}{2}} \right) \quad (5)$$

com $R = 6\,371\,000$ m (raio terrestre). Para o peso `length` essa h é diretamente admissível. Para `travel_time` dividimos h pela velocidade máxima observada na rede, preservando admissibilidade.

E. Dijkstra Bidirecional (Algoritmo da Literatura)

Algoritmo da literatura selecionado pelo grupo. Executa duas buscas Dijkstra em paralelo: uma *forward* partindo de A e outra *backward* partindo de B . Quando as fronteiras se encontram, o caminho ótimo é montado. O critério de parada

$$d_f(\text{top}_f) + d_b(\text{top}_b) \geq \text{melhor_distância} \quad (6)$$

garante otimalidade. Em pares longos, expande aproximadamente metade dos nós que Dijkstra completo expandiria. Esse princípio é a base de motores modernos como OSRM e Valhalla.

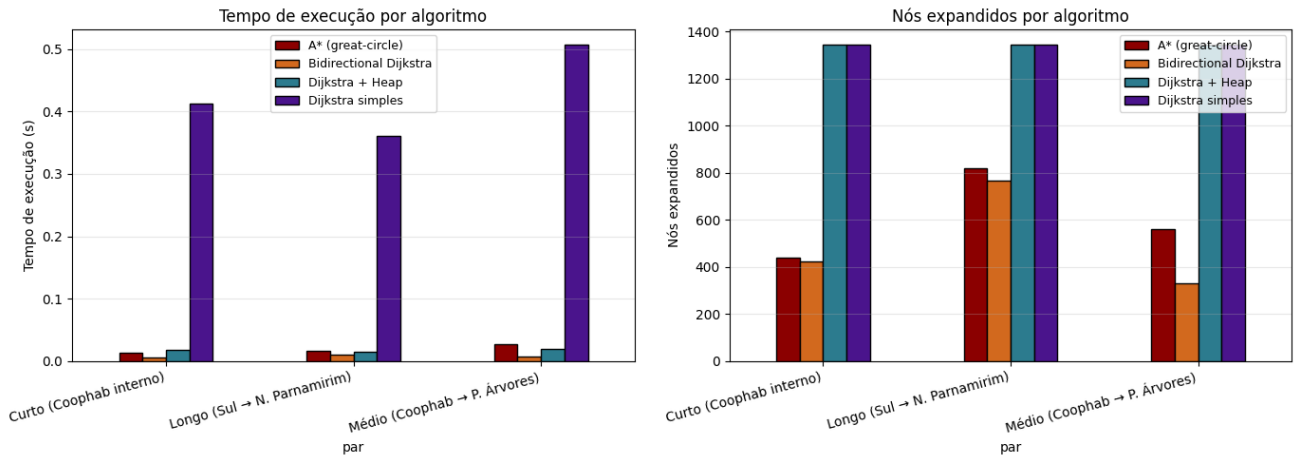


Figura 2. Comparação dos quatro algoritmos sobre o peso `travel_time_traffic`: tempo de execução (esquerda) e nós expandidos (direita).

F. Validação

Os quatro algoritmos foram validados contra `nx.shortest_path_length` do NetworkX em pares aleatórios. Em todos os testes, as distâncias coincidiram com erro inferior a 10^{-2} m.

IV. EXPERIMENTOS

A. Pares $A \rightarrow B$

Selecionamos três pares representativos, com nós identificados via `ox.distance.nearest_nodes`:

- **Curto** (Coophab interno): 2229 m em linha reta;
- **Médio** (Coophab → P. Árvores): 1449 m;
- **Longo** (Sul → N. Parnamirim): 3209 m, cruzando a Av. Olavo Montenegro.

O par *Longo* é especialmente revelador porque atravessa o **bridge node 3454188414 (Olavo Montenegro)** — que é um ponto de maior *betweenness centrality* da região.

B. Configurações Experimentais

Testamos $X \in \{100, 300, 500\}$ metros, gerando $3 \times 3 = 9$ configurações $\times$ 5 cenários = 45 execuções. Para cada execução coletamos: P escolhido, distância caminhada, tempo de caminhada (real e efetivo), distância dirigida, tempo de carro, tempo total, custo monetário e ganhos (segundos e R\$).

V. RESULTADOS

A. Eficiência dos Algoritmos

A Fig. 2 compara tempo de execução e nós expandidos. Sobre `travel_time_traffic`:

- Dijkstra simples consome $\sim 0,4$ s e expande todos os 1342 nós;
- Dijkstra + Heap reduz o tempo em $25\times$ ($\sim 0,02$ s) com o mesmo número de nós expandidos;
- A* expande de 40% a 70% menos nós, dependendo do par;
- Dijkstra Bidirecional é **competitivo com A*** e às vezes melhor (par Médio: 330 nós vs. 559 do A*).

B. Rotas Resultantes

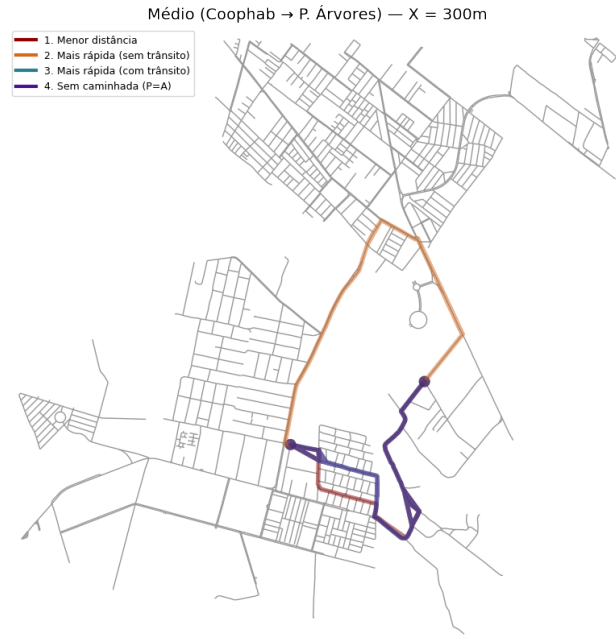


Figura 3. Rotas do par Médio (Coophab → P. Árvores) com $X = 300$ m. A rota mais rápida sem trânsito (laranja) faz um desvio expressivo pelo norte; as rotas com trânsito e sem caminhada convergem para o caminho central.

A Fig. 3 mostra as rotas geradas para o par Médio. A rota *sem trânsito* (laranja) faz um desvio de quase 1 km pelo norte usando vias arteriais; já a rota *com trânsito* (cyan, sobreposta) e a rota *sem caminhada* (roxa) preferem o caminho central pelas vias residenciais. A divergência ilustra o impacto do trânsito sobre a decisão ótima.

C. Ganho de Caminhar (Tempo)

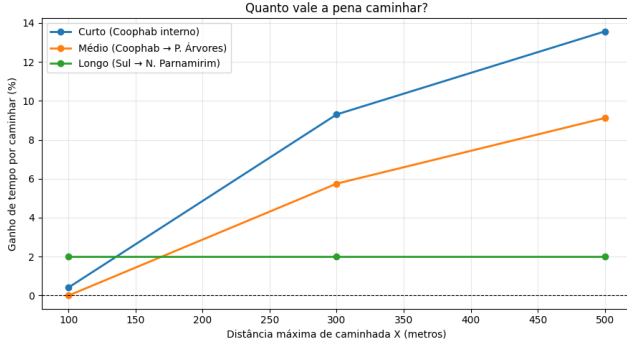


Figura 4. Ganho percentual de tempo ao caminhar até X metros, comparado a $P = A$. Curto e Médio crescem monotonicamente; Longo satura em $\sim 2\%$.

A Fig. 4 mostra como o ganho percentual de tempo varia com X . Para os pares Curto e Médio, o ganho cresce monotonicamente, atingindo **13,6% (75 segundos)** e **9,1% (67 segundos)** respectivamente em $X = 500$ m. O par Longo, em contraste, satura em $\sim 2\%$ após apenas 56 m de caminhada — uma saturação topológica detalhada na Seção VI.

D. Economia Monetária

A Fig. 5 (esquerda) mostra que a economia em reais acompanha o ganho de tempo, atingindo **R\$ 1,19** para o par Curto em $X = 500$ m. À direita, comparamos o custo absoluto da corrida entre os cenários 2, 3 e 4. Um achado não-óbvio: **no par Médio, o cenário “mais rápido sem trânsito” custa R\$ 17,10 — mais caro que o cenário “com trânsito” (R\$ 13,65)**. Isso ocorre porque a rota mais rápida em vias livres é também mais longa em quilômetros; sob trânsito, o algoritmo prefere uma rota mais curta em km que economiza dinheiro.

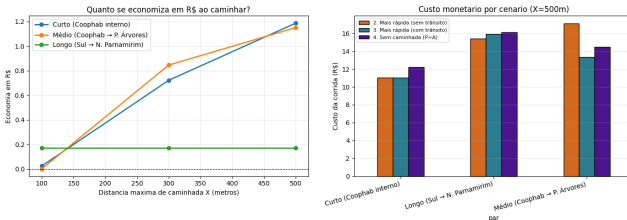


Figura 5. Economia em R\$ ao caminhar (esquerda) e custo absoluto da corrida por cenário em $X = 500$ m (direita). Note que no par Médio o cenário 2 (sem trânsito) é *mais caro* que o cenário 3 (com trânsito) — trade-off entre tempo e dinheiro.

VI. DISCUSSÃO CRÍTICA

A. Trade-off Tempo $\times$ Dinheiro

O caso do par Médio ilustra que minimizar tempo nem sempre coincide com minimizar custo. Aplicativos reais poderiam oferecer ao usuário a escolha explícita entre “rota rápida” e “rota econômica”, como já fazem alguns serviços (Uber Comfort vs UberX) — mas raramente expondo a métrica de R\$/min ao usuário.

B. Saturação Topológica do Par Longo

O par Longo cruza a Av. Olavo Montenegro, um corredor crítico. Caminhar mais de 56 m a partir do ponto A não traz benefício adicional, pois todos os P candidatos convergem para o mesmo ponto de travessia da avenida. Esse achado conecta-se diretamente ao *bridge node* 3454188414, identificado via *betweenness centrality* no trabalho de Unidade 2. Em termos práticos: **a topologia da rede impõe um limite natural** ao benefício da caminhada quando A está distante de um cruzamento crítico.

C. Limitações da Modelagem

- 1) **Trânsito sintético**: embora calibrado por tipo de via, não reflete dados reais.
- 2) **Não-direcionalidade**: convertemos para grafo simples não-direcionado, ignorando mão única.
- 3) **Velocidade de caminhada constante** (5 km/h): ignora ladeiras, semáforos e fluxo de pedestres.
- 4) **P livre**: na prática, motoristas têm restrições de embarque (vias bloqueadas, segurança).

D. Caminhos para Aplicações Reais

Sugerimos integrar: (i) dados de tráfego em tempo real; (ii) grafo composto (*drive + walk + transporte público*); (iii) restrição de P a pontos válidos de embarque; (iv) algoritmos de pré-processamento como *Contraction Hierarchies* para consultas em milissegundos em redes nacionais.

VII. CONCLUSÃO

Apresentamos o **RideSmart**, uma modelagem completa do problema de escolher o melhor ponto de embarque em aplicações de mobilidade urbana, integrando malha viária real, trânsito sintético, custo monetário e, crucialmente, o paralelismo entre caminhada e tempo de busca do motorista. Quatro algoritmos clássicos foram implementados *do zero* e comparados, evidenciando ganhos progressivos: Heap ($25\times$ sobre Dijkstra simples), A* (até 70% menos nós expandidos), Bidirectional (competitivo com A* sem depender de heurística).

Os resultados mostram que caminhar até alguns centenas de metros pode economizar **13,6% do tempo e R\$ 1,19** de uma corrida típica de 3 km. A análise revelou um *trade-off* não-trivial entre tempo e dinheiro (uma rota mais rápida pode ser mais cara), e um efeito de saturação topológica quando A se encontra distante de pontos críticos da rede (*bridge nodes*).